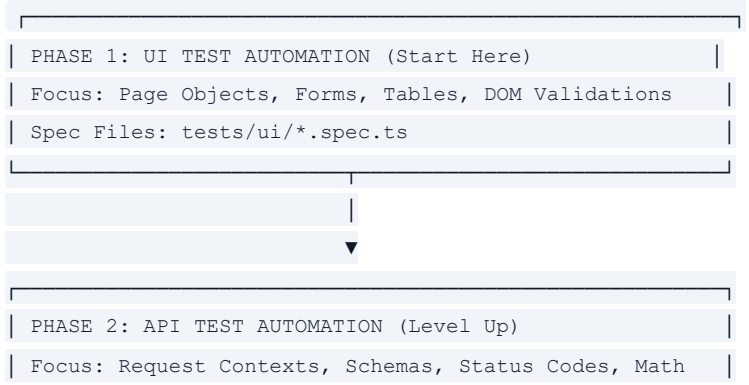
ParaBank — Test Case Specification

Field	Value
Project	ParaBank API + UI + Hybrid Automation Framework
Application Under Test (AUT)	ParaBank (Parasoft demo banking app)
UI Base URL	`https://parabank.parasoft.com/parabank`
REST Base URL	`https://parabank.parasoft.com/parabank/services/bank`
Tooling	Playwright Test, TypeScript, `request.newContext()`, `page.route()`
Execution Flow	**Phase 1: UI Testing → Phase 2: API Testing → Phase 3: Hybrid E2E & Mocking**
Document Version	2.0 (Progressive Learning & Implementation Flow)
Status	Active – Ready for Automation

---

1. Execution Flow & Automation Roadmap

To allow gradual implementation and learning, the test cases are organized sequentially:



```
| Spec Files: tests/api/*.api.spec.ts                    |
└──────────────────────────┬─────────────────────────────┘
                           |
                           ▼
┌──────────────────────────────────────────────────────────┐
| PHASE 3: HYBRID E2E & RESILIENCE (Mastery)               |
| Focus: API Setup ➔ UI Action ➔ API Verification          |
|        API Mocking via page.route()                      |
| Spec Files: tests/hybrid/*.e2e.spec.ts                   |
└──────────────────────────────────────────────────────────┘
```

---

2. Conventions & Standards

2.1 Test Case ID Structure

`TC-<MODULE>-<LAYER>-<NNN>`

- **Module**: `AUTH` (Login/Auth), `REG` (Registration), `ACC` (Accounts), `TRF` (Transfer), `BILL` (Bill Pay), `TXN` (Transactions), `USR` (Profile), `LOAN` (Loans)
- **Layer**: `UI` (Pure UI), `API` (Pure API), `HYB` (Hybrid E2E), `MCK` (Mocking via `page.route()`)

2.2 Classification Tags

- `Functional – Positive`: Standard happy path
- `Functional – Negative`: Input validation and error messaging
- `Boundary`: Zero, negative, decimal, and extreme values
- `Business Rule`: Domain mathematics, balances, double-entry ledger
- `Contract / Schema`: Payload structure, headers, data types
- `Security`: IDOR, authorization, SQLi/XSS injection
- `Resilience`: Front-end error handling when backend returns 4xx/5xx or timeouts
- `Performance (SLA)`: API response latency (< 2000 ms)

2.3 Priority Levels

- **P1 – Critical / Smoke**: Must pass for any build or deployment.
- **P2 – High**: Core business logic and validation.

- **P3 – Medium**: Edge scenarios, boundary conditions, extended features.

IMPORTANT: **Handling ParaBank Quirks (`@known-defect`):** ParaBank is a demo application and deliberately lacks some production banking constraints (e.g., negative transfers, overdraft limits, strict IDOR authorization). These test cases specify the **true industry banking standard**. When automating, document the deviation using `@known-defect` and Playwright's `test.fail()` to showcase professional defect management.

💻 PHASE 1: UI TEST AUTOMATION

Start here to build your Page Object Models (`pages/*.ts`) and master locator strategies, form interactions, navigation, and visual assertions.

1.1 Authentication & Navigation UI (`tests/ui/login.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-AUTH-UI-001`	Valid login	Functional – Positive	P1	Registered user	1) Open `index.htm` 2) Enter username and password 3) Click **Log In**	Redirected to `overview.htm`. Heading **Accounts Overview** visible.
`TC-AUTH-UI-002`	Verify dashboard after login	Functional – Positive	P1	Logged in	1) Inspect left panel and main area	Text **Welcome {First} {Last}** displayed. Account table visible. Full menu: Open Account, Overview, Transfer, Bill Pay, Find Transactions, Update Profile, Request Loan, Logout.
`TC-AUTH-UI-003`	Login with invalid username	Functional – Negative	P1	None	1) Enter unknown username + any password 2) Click **Log In**	Error message: **The username and password could not be verified.** User remains logged out.
`TC-AUTH-UI-004`	Login with invalid password	Functional – Negative	P1	Registered user	1) Enter valid username + wrong password 2) Click **Log In**	Error message: **The username and password could not be verified.**
`TC-AUTH-UI-005`	Login with empty credentials	Functional – Negative	P1	None	1) Leave both fields empty 2) Click **Log In**	Error message: **Please enter a username and password.**
`TC-AUTH-UI-006`	Login with empty password only	Functional – Negative	P2	None	1) Enter username, leave password blank 2) Click **Log In**	Error message: **Please enter a username and password.**

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-AUTH-UI-007`	Login with empty username only	Functional – Negative	P2	None	1) Enter password, leave username blank 2) Click **Log In**	Error message: **Please enter a username and password.**
`TC-AUTH-UI-008`	Logout workflow	Functional – Positive	P1	Logged in	1) Click **Log Out** in menu	Redirected to `index.htm`. Login form visible. Welcome text cleared.
`TC-AUTH-UI-009`	Protected page access after logout	Security	P1	Logged out	1) Directly navigate to `overview.htm` 2) Click browser Back button	Session revoked. Login form or error page displayed; account data not shown.
`TC-AUTH-UI-010`	Password field masking	Security	P3	None	1) Inspect password input field	HTML attribute `type="password"` present. Input characters obscured.
`TC-AUTH-UI-011`	Submit login using Enter key	Navigation / Usability	P3	Registered user	1) Enter username and password 2) Press **Enter** key	Form submitted successfully; navigated to `overview.htm`.

1.2 User Registration UI (`tests/ui/profile.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-REG-UI-001`	Register new user via UI	Functional – Positive	P1	Generated user data	1) Click **Register** link 2) Fill all personal details 3) Click **Register**	Heading **Welcome {username}**. Text: "Your account was created successfully. You are now logged in."
`TC-REG-UI-002`	Register with completely empty form	Functional – Negative	P1	None	1) Navigate to register page 2) Click **Register**	Validation errors appear for all required fields: First Name, Last Name, Address, City, State, Zip, SSN, Username, Password.
`TC-REG-UI-003`	Password confirmation mismatch	Functional – Negative	P1	None	1) Fill valid form 2) Password: `Secret123!` 3) Confirm: `Mismatch999!` 4) Submit	Error displayed: **Passwords did not match.**
`TC-REG-UI-004`	Duplicate username registration	Functional – Negative	P1	Existing user	1) Fill form using an already registered username 2) Submit	Error displayed: **This username already exists.**

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-REG-UI-005	Login with freshly registered user	Functional – Positive	P1	User registered	1) Log out from registration 2) Log in with new credentials	Successfully logged in; Accounts Overview shows 1 default account.

1.3 Accounts Management UI (`tests/ui/accounts.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-ACC-UI-001	View Accounts Overview table	Functional – Positive	P1	Logged in	1) Click **Accounts Overview**	Table contains headers: **Account, Balance*, Available Amount**. Row for each account and a **Total** row.
TC-ACC-UI-002	Verify Total equals sum of balances	Business Rule	P2	Logged in, 2+ accounts	1) Parse all balance column values 2) Sum mathematically 3) Read Total row	Total matches sum of all individual balances within $0.01.
TC-ACC-UI-003	Balance currency formatting	Navigation / Usability	P3	Logged in	1) Inspect balance values in table	Format conforms strictly to `^-?\$[\d,]+\.\d{2}$`.
TC-ACC-UI-004	Open new CHECKING account	Functional – Positive	P1	Logged in	1) Click **Open New Account** 2) Select **CHECKING** 3) Select source account 4) Click **Open New Account**	Heading **Account Opened!**. Text: "Congratulations, your account is now open." New account number link displayed.
TC-ACC-UI-005	Open new SAVINGS account	Functional – Positive	P1	Logged in	1) Repeat flow selecting **SAVINGS**	Heading **Account Opened!** with a unique new account number.
TC-ACC-UI-006	Account type dropdown options	Navigation / Usability	P3	Logged in	1) Open **Open New Account** page 2) Inspect dropdown	Options present: `CHECKING` and `SAVINGS`.
TC-ACC-UI-007	Verify account details page	Functional – Positive	P1	Account opened	1) Click the newly created account number link	**Account Details** shows matching Account Number, Account Type, and Balance.
TC-ACC-UI-008	New account reflected in overview	Functional – Positive	P2	Account opened	1) Navigate to **Accounts Overview**	New account row is listed in the main table with updated total.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-ACC-UI-009`	Navigation between multiple accounts	Navigation / Usability	P2	Logged in, 2+ accounts	1) Click Account #1 link ➔ check details 2) Back to overview 3) Click Account #2 link ➔ check details	Each details page displays the specific account number clicked.
`TC-ACC-UI-010`	Account activity period filter	Functional – Positive	P3	Account with data	1) Open account details 2) Set Activity Period = All, Type = Debit 3) Click **Go**	Activity table updates to show only Debit entries.
`TC-LOAN-UI-001`	Request loan – approved	Functional – Positive	P3	Logged in	1) Click **Request Loan** 2) Amount: 500, Down Payment: 50 3) Click **Apply Now**	Heading **Loan Request Processed**. Status: **Approved**. New account number generated.
`TC-LOAN-UI-002`	Request loan – denied	Business Rule	P3	Logged in	1) Amount: 500000, Down Payment: 10 2) Click **Apply Now**	Heading **Loan Request Processed**. Status: **Denied**.

1.4 Fund Transfer UI (`tests/ui/transfer.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-TRF-UI-001`	Valid transfer via UI	Functional – Positive	P1	Logged in, 2 accounts	1) Click **Transfer Funds** 2) Amount: 25.00 3) Select From Account A and To Account B 4) Click **Transfer**	Heading **Transfer Complete!**. Text: "$25.00 has been transferred from account #A to account #B."
`TC-TRF-UI-002`	From / To account dropdowns populated	Functional – Positive	P2	Logged in	1) Open **Transfer Funds**	Both From and To dropdowns contain all of the user's active account numbers.
`TC-TRF-UI-003`	Transfer with empty amount	Functional – Negative	P1	Logged in	1) Leave amount blank 2) Select accounts 3) Click **Transfer**	Validation error displayed (e.g., **The amount cannot be empty.**). No confirmation.
`TC-TRF-UI-004`	Transfer with non-numeric amount	Functional – Negative	P2	Logged in	1) Enter amount: `abc` 2) Click **Transfer**	Validation error displayed (e.g., **Please enter a valid amount.**).
`TC-TRF-UI-005`	Transfer with zero amount	Boundary / Negative	P2	Logged in	1) Enter amount: `0` 2) Click **Transfer**	Form rejected. `@known-defect` if accepted by AUT.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-TRF-UI-006	Transfer with negative amount	Functional – Negative	P2	Logged in	1) Enter amount: `-10` 2) Click **Transfer**	Form rejected. `@known-defect` if money flow reverses.
TC-TRF-UI-007	Transfer to the same account	Business Rule / Negative	P3	Logged in	1) Select same account for From and To 2) Click **Transfer**	Validation warning or error. `@known-defect` if processed.

1.5 Bill Payment UI (`tests/ui/billPayment.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-BILL-UI-001	Valid bill payment submission	Functional – Positive	P1	Logged in, funded account	1) Click **Bill Pay** 2) Enter payee details (Name, Address, City, State, Zip, Phone, Account, Verify Account) 3) Amount: 30.00 4) Click **Send Payment**	Heading **Bill Payment Complete**. Text confirms payment of $30.00 to payee from selected account.
TC-BILL-UI-002	Submit completely empty bill pay form	Functional – Negative	P1	Logged in	1) Navigate to Bill Pay 2) Click **Send Payment**	Required errors: Payee name, Address, City, State, Zip Code, Phone, Account, and Amount required.
TC-BILL-UI-003	Account and Verify Account mismatch	Functional – Negative	P1	Logged in	1) Fill form 2) Account: `11111`, Verify Account: `22222` 3) Submit	Error message: **The account numbers do not match.**
TC-BILL-UI-004	Non-numeric account number	Functional – Negative	P2	Logged in	1) Enter Account: `XYZ` 2) Submit	Validation error: **Please enter a valid number.**
TC-BILL-UI-005	Non-numeric amount	Functional – Negative	P2	Logged in	1) Enter Amount: `test` 2) Submit	Validation error: **Please enter a valid amount.**
TC-BILL-UI-006	From account dropdown verification	Functional – Positive	P3	Logged in	1) Inspect From Account dropdown	Displays all accounts belonging to the current user.
TC-BILL-UI-007	XSS prevention in payee input	Security	P3	Logged in	1) Enter payee name: `<script>alert('xss')</script>` 2) Submit	Input is properly escaped in confirmation text; no browser dialog executed.

1.6 Transactions History & Search UI (`tests/ui/transactions.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-TXN-UI-001`	View account activity table	Functional – Positive	P1	Account with transactions	1) Accounts Overview ➔ click an account link	Activity table displayed with columns: **Date, Transaction, Debit (-), Credit (+)**.
`TC-TXN-UI-002`	View transaction details	Functional – Positive	P1	Account with transactions	1) Click transaction description link	**Transaction Details** page opens showing: ID, Date, Description, Type, Amount.
`TC-TXN-UI-003`	Verify transaction details match row	Functional – Positive	P2	Account with transactions	1) Note Date, Type, Amount on table row 2) Open details page	Values on details page match the activity row exactly.
`TC-TXN-UI-004`	Search transactions by Transaction ID	Functional – Positive	P2	Known transaction ID	1) Click **Find Transactions** 2) Select account 3) Enter ID ➔ click **Find Transactions**	Results table contains exactly the single matching transaction.
`TC-TXN-UI-005`	Search transactions by Date	Functional – Positive	P2	Transaction made today	1) Find Transactions ➔ enter today's date (MM-DD-YYYY) ➔ search	Results table displays all transactions executed on that date.
`TC-TXN-UI-006`	Search transactions by Date Range	Functional – Positive	P2	Transactions in range	1) Enter From Date and To Date ➔ search	All returned transactions fall strictly within the specified date range.
`TC-TXN-UI-007`	Search transactions by Amount	Functional – Positive	P2	Known amount	1) Enter amount: `25.00` ➔ search	Results table displays only transactions matching $25.00.
`TC-TXN-UI-008`	Search with non-numeric Transaction ID	Functional – Negative	P2	Logged in	1) Enter ID: `abc` ➔ search	Validation error: **Invalid transaction ID.**
`TC-TXN-UI-009`	Search with invalid date format	Functional – Negative	P3	Logged in	1) Enter date: `99/99/2026` ➔ search	Validation error: **Invalid date format.**
`TC-TXN-UI-010`	Search with invalid amount	Functional – Negative	P3	Logged in	1) Enter amount: `xyz` ➔ search	Validation error: **Invalid amount.**

1.7 User Profile & Contact Info UI (`tests/ui/profile.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-USR-UI-001`	Update Profile form pre-filled	Functional – Positive	P2	Logged in	1) Click **Update Contact Info**	Fields (First Name, Last Name, Address, City, State, Zip, Phone) are pre-populated with current profile data.
`TC-USR-UI-002`	Update profile details	Functional – Positive	P1	Logged in	1) Modify Street Address and Phone 2) Click **Update Profile**	Heading **Profile Updated**. Text confirms update success.
`TC-USR-UI-003`	Update profile with missing required field	Functional – Negative	P2	Logged in	1) Clear First Name field 2) Click **Update Profile**	Validation error: **First name is required.**
`TC-USR-UI-004`	Profile updates persist across sessions	Functional – Positive	P3	Profile updated	1) Log out 2) Log in again 3) Open Update Contact Info	Updated address and phone number remain saved.

🔌 PHASE 2: API TEST AUTOMATION

Level up to fast backend testing: validate HTTP contracts, JSON/XML schemas, business math, status codes, headers, and security boundaries using `request.newContext()`.

2.1 Authentication API (`tests/api/auth.api.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-AUTH-API-001`	Authenticate with valid credentials	Functional – Positive	P1	Registered user	1) `GET /login/{username}/{password}` with `Accept: application/json`	Status **200 OK**. Response body contains customer object with correct ID and names.
`TC-AUTH-API-002`	Validate login JSON response schema	Contract / Schema	P1	Registered user	1) Call login endpoint 2) Validate JSON schema	Fields present: `id` (int), `firstName` (str), `lastName` (str), `address` (obj), `phoneNumber` (str), `ssn` (str).

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-AUTH-API-003	Validate response headers	Contract / Schema	P2	Registered user	1) Call login endpoint 2) Read headers	Header `Content-Type` contains `application/json`.
TC-AUTH-API-004	Content negotiation for XML	Contract / Schema	P3	Registered user	1) Call login endpoint with `Accept: application/xml`	Status **200 OK**. Header contains `xml`. Body parses with `<customer>` root.
TC-AUTH-API-005	Login response latency SLA	Performance (SLA)	P2	Registered user	1) Measure request duration	Response time is **< 2000 ms**.
TC-AUTH-API-006	Login with unknown username	Functional – Negative	P1	None	1) `GET /login/fake_user_999/{pass}`	Status **400 Bad Request**. Body: `Invalid username and/or password`.
TC-AUTH-API-007	Login with incorrect password	Functional – Negative	P1	Registered user	1) `GET /login/{user}/WrongPassword!`	Status **400 Bad Request**. Same error message (avoids username enumeration).
TC-AUTH-API-008	Login with missing credentials	Functional – Negative	P2	None	1) `GET /login//`	Status **400** or **404**. No unhandled 500 error.
TC-AUTH-API-009	SQL injection payload in login	Security	P2	None	1) `GET /login/' OR '1'='1/{pass}` (URL-encoded)	Status **400 Bad Request**. No SQL errors or stack traces leaked.
TC-AUTH-API-010	Method Not Allowed validation	Contract / Schema	P3	None	1) `POST /login/{user}/{pass}`	Status **405 Method Not Allowed** (or 404). Never 200.

2.2 Accounts API (`tests/api/accounts.api.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-ACC-API-001	Fetch accounts for customer	Functional – Positive	P1	Customer ID	1) `GET /customers/{id}/accounts`	Status **200 OK**. Non-empty array of account objects.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-ACC-API-002	Validate account schema & data types	Contract / Schema	P1	Customer ID	1) Validate each account in array	Fields: `id` (number), `customerId` (number), `type` (CHECKING/SAVINGS/LOAN), `balance` (number).
TC-ACC-API-003	Get account details by ID	Functional – Positive	P1	Known account ID	1) `GET /accounts/{accountId}`	Status **200 OK**. Returned `id` equals requested ID.
TC-ACC-API-004	Verify customer account ownership	Business Rule	P1	Logged in customer	1) Call get account 2) Inspect `customerId`	Returned `customerId` matches the authenticated customer's ID.
TC-ACC-API-005	Create CHECKING account via API	Functional – Positive	P1	Funded account	1) `POST /createAccount?customerId={cId}&newAccountType=0&fromAccountId={aId}`	Status **200 OK**. New account object returned with `type: "CHECKING"`.
TC-ACC-API-006	Create SAVINGS account via API	Functional – Positive	P1	Funded account	1) `POST /createAccount` with `newAccountType=1`	Status **200 OK**. New account object returned with `type: "SAVINGS"`.
TC-ACC-API-007	Minimum opening deposit debit/credit	Business Rule	P1	Initial balance S	1) Create account from source account 2) Check both balances	New account balance = opening deposit ($100.00). Source account balance = S − $100.00.
TC-ACC-API-008	Newly created account in list	Business Rule	P2	Account created	1) `GET /customers/{id}/accounts`	New account ID is present in the customer's account list.
TC-ACC-API-009	Balance decimal precision	Business Rule	P2	Any account	1) Inspect account balance	Balance has at most 2 decimal places (`balance * 100` is integer).
TC-ACC-API-010	Get non-existent account ID	Functional – Negative	P1	None	1) `GET /accounts/999999999`	Status **400 Bad Request**. Error message: `Could not find account #999999999`.
TC-ACC-API-011	Get account with non-numeric ID	Functional – Negative	P2	None	1) `GET /accounts/invalid_id`	Status **400** or **404**.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-ACC-API-012	Get accounts for non-existent customer	Functional – Negative	P2	None	1) `GET /customers/999999999/accounts`	Status **400 Bad Request**.
TC-ACC-API-013	Create account with invalid type code	Functional – Negative	P2	Valid customer	1) `POST /createAccount` with `newAccountType=99`	Status **400 Bad Request**. No account created.
TC-ACC-API-014	Create account missing parameters	Functional – Negative	P2	Valid customer	1) `POST /createAccount` with no query parameters	Status **400 Bad Request**.
TC-ACC-API-015	Create account using another user's funds	Security / IDOR	P2	User A and User B	1) User A creates account using User B's `fromAccountId`	Request rejected with 4xx. `@known-defect` if accepted by AUT.
TC-ACC-API-016	Unauthorized accounts request	Security	P1	Clean context	1) Request account data with no session or auth token	Status **401 Unauthorized**. `@known-defect` if data is exposed.
TC-ACC-API-017	Accounts API response headers & SLA	Contract / SLA	P2	Valid account	1) Verify latency and headers	`Content-Type: application/json`. Duration < 2000 ms.
TC-LOAN-API-001	Request loan – approved API	Functional – Positive	P3	Customer & account	1) `POST /requestLoan?customerId={cId}&amount=500&downPayment=50&fromAccountId={aId}`	Status **200 OK**. Response: `approved: true`. New LOAN account ID returned.
TC-LOAN-API-002	Request loan – denied API	Business Rule	P3	Customer & account	1) `POST /requestLoan` with down payment exceeding balance	Status **200 OK**. Response: `approved: false`.

2.3 Fund Transfer API (`tests/api/transfer.api.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-TRF-API-001`	Valid transfer between own accounts	Functional – Positive	P1	2 accounts, source funded	1) `POST /transfer?fromAccountId={A}&toAccountId={B}&amount=50`	Status **200 OK**. Body: `Successfully transferred $50 from account #A to account #B`.
`TC-TRF-API-002`	Source balance debit verification	Business Rule	P1	Source balance S	1) Transfer X 2) `GET /accounts/{A}`	Source balance equals exactly S − X.
`TC-TRF-API-003`	Destination balance credit verification	Business Rule	P1	Destination balance D	1) Transfer X 2) `GET /accounts/{B}`	Destination balance equals exactly D + X.
`TC-TRF-API-004`	Conservation of total balances	Business Rule	P2	Balances S and D	1) Transfer X 2) Sum new balances	$(S' + D') = (S + D)$. Zero money gained or lost.
`TC-TRF-API-005`	Transfer with fractional cents amount	Boundary	P2	Funded source	1) Transfer `amount=12.34`	Status **200 OK**. Balances adjusted by exactly $12.34 without float drift.
`TC-TRF-API-006`	Double-entry transaction creation	Business Rule	P1	2 accounts	1) Transfer X 2) Fetch transactions for A and B	Account A has Debit of X ("Funds Transfer Sent"). Account B has Credit of X ("Funds Transfer Received").
`TC-TRF-API-007`	Transfer with zero amount	Boundary / Negative	P2	2 accounts	1) Transfer `amount=0`	Status **400 Bad Request**. `@known-defect` if accepted.
`TC-TRF-API-008`	Transfer with negative amount	Functional – Negative	P1	2 accounts	1) Transfer `amount=-50`	Status **400 Bad Request**. `@known-defect` if accepted.
`TC-TRF-API-009`	Transfer with empty amount parameter	Functional – Negative	P2	2 accounts	1) Transfer `amount=`	Status **400 Bad Request**.
`TC-TRF-API-010`	Transfer with non-numeric amount	Functional – Negative	P2	2 accounts	1) Transfer `amount=abc`	Status **400 Bad Request**.
`TC-TRF-API-011`	Transfer with invalid source account	Functional – Negative	P1	Valid destination	1) Transfer `fromAccountId=999999999`	Status **400 Bad Request**. Error: `Could not find account #999999999`.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-TRF-API-012	Transfer with invalid destination account	Functional – Negative	P1	Valid source	1) Transfer `toAccountId=999999999`	Status **400 Bad Request**. Source balance unchanged.
TC-TRF-API-013	Transfer from and to the same account	Business Rule / Negative	P2	1 account	1) Transfer `fromAccountId={A}&toAccountId={A}`	Status **400 Bad Request**. `@known-defect` if accepted.
TC-TRF-API-014	Transfer exceeding available balance	Business Rule / Negative	P1	Source balance S	1) Transfer `amount=S+5000`	Rejected with 4xx. Balance unchanged. `@known-defect` if overdraft allowed.
TC-TRF-API-015	Transfer missing query parameters	Functional – Negative	P2	None	1) `POST /transfer` with no parameters	Status **400 Bad Request**.
TC-TRF-API-016	Transfer with excessive decimal places	Boundary	P3	Funded source	1) Transfer `amount=10.999`	Rejected or rounded cleanly to 2 decimals.
TC-TRF-API-017	Transfer from another customer's account	Security / IDOR	P2	Customer A and B	1) Customer A transfers from Customer B's account	Status **403 Forbidden** / 400. `@known-defect` if accepted.
TC-TRF-API-018	Unauthorized transfer request	Security	P1	Clean context	1) `POST /transfer` without authentication	Status **401 Unauthorized**. `@known-defect` if accepted.
TC-TRF-API-019	Transfer response SLA	Performance (SLA)	P3	Funded source	1) Measure transfer latency	Latency < 2000 ms.

2.4 Transactions API (`tests/api/transactions.api.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-TXN-API-001	Get transactions for account	Functional – Positive	P1	Active account	1) `GET /accounts/{id}/transactions`	Status **200 OK**. Array of transaction objects.
TC-TXN-API-002	Validate transaction schema & types	Contract / Schema	P1	Active account	1) Validate transaction fields	Fields: `id` (int), `accountId` (int), `type` (Credit/Debit), `date` (epoch ms), `amount` (float), `description` (str).

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-TXN-API-003	Get individual transaction by ID	Functional – Positive	P1	Known transaction ID	1) `GET /transactions/{id}`	Status **200 OK**. Payload matches account transaction list entry.
TC-TXN-API-004	Transaction account association	Business Rule	P2	Active account	1) Fetch account transactions 2) Check each `accountId`	Every item's `accountId` equals the requested account ID.
TC-TXN-API-005	Query transactions by specific amount	Business Rule	P1	Transfer X done	1) `GET /accounts/{id}/transactions/amount/{X}`	Returns matching transactions with `amount` = X.
TC-TXN-API-006	Query transactions on specific date	Functional – Positive	P2	Transfer done today	1) `GET .../transactions/onDate/{MM-DD-YYYY}`	Returns all transactions recorded on that calendar date.
TC-TXN-API-007	Query transactions within date range	Functional – Positive	P2	Transactions exist	1) `GET .../fromDate/{d1}/toDate/{d2}`	All returned records have dates falling within [d1, d2].
TC-TXN-API-008	Get transaction with non-existent ID	Functional – Negative	P1	None	1) `GET /transactions/999999999`	Status **400 Bad Request**. Error: `Could not find transaction #...`.
TC-TXN-API-009	Get transactions for invalid account	Functional – Negative	P2	None	1) `GET /accounts/999999999/transactions`	Status **400 Bad Request**.
TC-TXN-API-010	Query transactions with invalid date format	Functional – Negative	P3	Valid account	1) `GET .../onDate/2026-99-99`	Status **400** or **404**.

2.5 Bill Payment API (`tests/api/transfer.api.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-BILL-API-001	Submit valid bill payment	Functional – Positive	P1	Funded account	1) `POST /billpay?accountId={id}&amount=45` with JSON payee payload	Status **200 OK**. Body contains `payeeName`, `amount: 45`, `accountId`.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-BILL-API-002	Account balance debited after bill pay	Business Rule	P1	Balance B	1) Execute bill pay of X 2) Get account	Balance equals exactly B − X.
TC-BILL-API-003	Bill pay creates transaction record	Business Rule	P2	Payment executed	1) Get account transactions	New Debit entry of amount X with description `Bill Payment to {payee}`.
TC-BILL-API-004	Bill pay from invalid account ID	Functional – Negative	P2	None	1) `POST /billpay?accountId=999999999&amount=20`	Status **400 Bad Request**.
TC-BILL-API-005	Bill pay missing JSON body payload	Functional – Negative	P2	Funded account	1) `POST /billpay?accountId={id}&amount=20` with no body	Status **400** or **415 Unsupported Media Type**.
TC-BILL-API-006	Bill pay with negative amount	Functional – Negative	P2	Funded account	1) `POST /billpay` with `amount=-50`	Status **400 Bad Request**. `@known-defect` if accepted.
TC-BILL-API-007	Pre-validate test account for bill pay	Functional – Positive	P2	Generated user	1) Create account 2) Assert account details	Account exists, status active, balance funded (> $0). Used by fixtures.

2.6 User Profile & Customers API (`tests/api/users.api.spec.ts`)

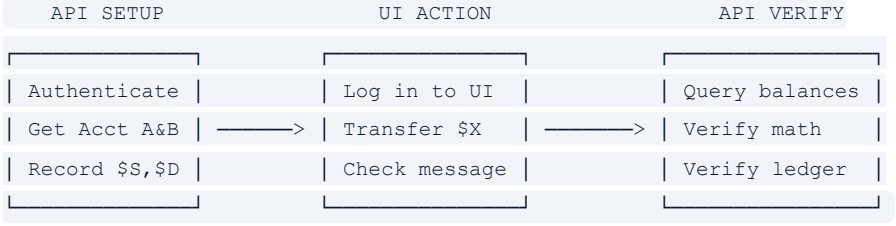
ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-USR-API-001	Register user via form POST	Functional – Positive	P1	Generated user	1) `POST /parabank/register.htm` (form-urlencoded) 2) `GET /login/{user}/{pass}`	Registration accepted; subsequent login returns 200 with customer ID.
TC-USR-API-002	Fetch customer details by Customer ID	Functional – Positive	P1	Customer ID	1) `GET /customers/{id}`	Status **200 OK**. Customer fields match registration data.
TC-USR-API-003	Update customer profile details	Functional – Positive	P1	Customer ID	1) `POST /customers/update/{id}?city=NewCity&phoneNumber=555-9999...`	Status **200 OK**. Success confirmation message returned.
TC-USR-API-004	Updated profile persists in database	Business Rule	P1	Profile updated	1) `GET /customers/{id}`	Response contains updated city and phone number; other fields unchanged.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-USR-API-005`	Prevent duplicate registration	Functional – Negative	P1	Existing user	1) Re-submit registration form with existing username	Response body contains: **This username already exists.**
`TC-USR-API-006`	Get non-existent customer ID	Functional – Negative	P1	None	1) `GET /customers/999999999`	Status **400 Bad Request**. Error: `Could not find customer #999999999`.
`TC-USR-API-007`	Get customer with non-numeric ID	Functional – Negative	P3	None	1) `GET /customers/invalid_user`	Status **400** or **404**.
`TC-USR-API-008`	Update profile missing required params	Functional – Negative	P2	Customer ID	1) `POST /customers/update/{id}` with empty parameters	Status **400 Bad Request**.
`TC-USR-API-009`	Ensure customer DELETE endpoint disabled	Contract / Security	P3	Customer ID	1) `DELETE /customers/{id}`	Status **404** or **405 Method Not Allowed**. Customer remains active.
`TC-USR-API-010`	IDOR protection on customer profile	Security	P2	Customer A & B	1) Authenticate as A 2) Request `GET /customers/{B.id}`	Status **403 Forbidden**. `@known-defect` if B's personal data is exposed.

🚀 PHASE 3: HYBRID WORKFLOWS & API MOCKING

The crown jewel of your portfolio: combine UI and API into fast, realistic hybrid workflows, and test front-end resilience against server failures using Playwright's `page.route()`.

3.1 Hybrid Fund Transfer Workflows (`tests/hybrid/transfer.e2e.spec.ts`)



ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-TRF-HYB-001	**Main Banking Flow: API Setup ➔ UI Transfer ➔ API Balance Verification**	Functional – Positive / Business Rule	P1	Fixtures: `authenticatedUser`, `sourceAccount`, `destinationAccount`	1) **API:** Authenticate & fetch Accounts A & B 2) **API:** Record opening balances S and D 3) **UI:** Navigate to Transfer Funds ➔ Transfer $X ➔ Submit 4) **UI:** Assert confirmation message 5) **API:** Fetch balances	UI confirms: **Transfer Complete!**. API confirms: Account A balance = $S - X$, Account B balance = $D + X$.
TC-TRF-HYB-002	Transfer between newly API-created accounts	Functional – Positive	P1	New customer	1) **API:** Create 2 new accounts 2) **UI:** Execute transfer between them 3) **API:** Verify balances	Both new accounts display correct updated balances via API.
TC-TRF-HYB-003	UI transfer logged in API transaction ledger	Business Rule	P1	Same as 001	1) **UI:** Perform transfer of amount $X 2) **API:** Query transaction history for both accounts	Account A has Debit transaction of $X; Account B has Credit transaction of $X dated today.
TC-TRF-HYB-004	Fractional decimal transfer precision E2E	Boundary	P2	Same as 001	1) **API:** Record balances 2) **UI:** Transfer `$12.34` 3) **API:** Assert balances	Balances change by exactly $12.34 without IEEE 754 floating-point errors.
TC-TRF-HYB-005	Failed UI transfer leaves backend balances intact	Functional – Negative	P1	Same as 001	1) **API:** Record balances 2) **UI:** Submit transfer with empty amount 3) **API:** Fetch balances	UI shows validation error. API confirms balances remain 100% unchanged.
TC-TRF-HYB-006	Multiple sequential UI transfers stay consistent	Business Rule	P2	Same as 001	1) **API:** Record balances 2) **UI:** Transfer $10, then $20, then $30 3) **API:** Fetch balances	Account A debited by exactly $60; Account B credited by exactly $60.
TC-TRF-HYB-007	API transfer immediately reflected in UI table	Functional – Positive	P2	Logged in UI session	1) **API:** Perform transfer of $X 2) **UI:** Reload Accounts Overview	Overview table displays updated balances matching backend API state.

3.2 Hybrid Bill Payment Workflows (`tests/hybrid/payment.e2e.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-BILL-HYB-001`	**API Seed Account ➔ UI Pay Bill ➔ API Balance & Transaction Check**	Functional – Positive / Business Rule	P1	Fixture: `testAccount`	1) **API:** Create funded account, record balance B 2) **UI:** Fill bill payment form ($X) ➔ Submit 3) **UI:** Assert confirmation message 4) **API:** Fetch balance and transactions	UI displays: **Bill Payment Complete**. API confirms balance = $B - X$, and new transaction `Bill Payment to {payee}` Debit X.
`TC-BILL-HYB-002`	Failed UI bill pay leaves backend balance intact	Functional – Negative	P2	Same as 001	1) **API:** Record balance 2) **UI:** Submit mismatched account numbers 3) **API:** Fetch balance	UI shows mismatch error. API balance remains unchanged; no transaction logged.
`TC-BILL-HYB-003`	Payment exceeding balance rejected	Business Rule / Negative	P2	Account balance B	1) **UI:** Submit bill pay of $B + 1000 2) **API:** Assert balance	UI displays error. Backend balance remains unchanged. `@known-defect` if overdraft allowed.

3.3 Hybrid Account & Profile Workflows (`tests/hybrid/account.e2e.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
`TC-USR-HYB-001`	**API Create User ➔ UI Login ➔ UI Update Profile ➔ API Verification**	Functional – Positive / Business Rule	P1	Generated user	1) **API:** Register new user 2) **UI:** Log in with credentials 3) **UI:** Update city & phone 4) **API:** `GET /customers/{id}`	UI shows **Profile Updated**. API confirms updated city and phone persisted in database.
`TC-USR-HYB-002`	API user data correctly populated in UI	Functional – Positive	P2	Generated user	1) **API:** Register user 2) **UI:** Log in 3) Inspect dashboard and profile form	UI welcome banner and contact form fields match API registration data verbatim.
`TC-USR-HYB-003`	UI registration verified via API	Functional – Positive	P2	Generated user	1) **UI:** Register new user 2) **API:** Call login & get customer	API returns 200 with customer data matching UI registration form inputs.
`TC-ACC-HYB-001`	API Account Creation ➔ UI Verification	Functional – Positive	P1	Authenticated user	1) **API:** Create SAVINGS account 2) **UI:** Refresh Accounts Overview ➔ open new account	New account visible in UI overview; details page displays matching account number, type, and opening balance.

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-ACC-HYB-002	UI Account Creation ➔ API Verification	Functional – Positive	P1	Logged in user	1) **UI:** Open new CHECKING account ➔ capture number 2) **API:** `GET /accounts/{id}`	API returns account with `type: "CHECKING"`, correct customer ID, and opening balance.
TC-ACC-HYB-003	Balance totals consistency between API and UI	Business Rule	P2	User with 2+ accounts	1) **API:** Sum balances across all accounts 2) **UI:** Read Total row in Accounts Overview	UI Total figure mathematically matches sum of balances from API.

3.4 Hybrid Transaction Verification (`tests/hybrid/transaction.e2e.spec.ts`)

ID	Test Case	Classification	Priority	Preconditions	Steps	Expected Result
TC-TXN-HYB-001	**API Transaction ➔ UI Search ➔ API Record Match**	Functional – Positive	P1	2 accounts	1) **API:** Execute transfer of unique amount $X (e.g., $19.87) 2) **UI:** Find Transactions by Amount $X 3) **UI:** Open transaction ➔ note ID 4) **API:** `GET /transactions/{id}`	UI displays transaction with amount $X and date today. API endpoint returns matching record.
TC-TXN-HYB-002	API transfer displays in UI Account Activity table	Functional – Positive	P2	2 accounts	1) **API:** Transfer $X 2) **UI:** Open Account Details ➔ inspect activity table	Top row displays today's date, "Funds Transfer Sent", and Debit -$X.
TC-TXN-HYB-003	UI transfer creates credit entry verified in API	Business Rule	P2	2 accounts	1) **UI:** Submit transfer of $X 2) **API:** Fetch destination account transactions	API returns Credit +$X transaction dated today.
TC-TXN-HYB-004	API bill payment searchable in UI by date	Functional – Positive	P3	Funded account	1) **API:** Submit bill pay of $X 2) **UI:** Find Transactions by Date ➔ search today	Results table lists "Bill Payment to {payee}" with amount $X.

3.5 UI Resilience — API Mocking with `page.route()`

Intercept network traffic (`**/services_proxy/bank/**`) to prove the UI handles server errors gracefully without unhandled exceptions or crashes.

ID	Test Case	Classification	Priority	Target Spec	Mock Setup	Action	Expected Result
`TC-TRF-MCK-001`	Transfer – Mocked 200 Success	Resilience	P2	transfer.spec.ts	`route.fulfill({ status: 200, body: 'Successfully transferred...' })`	Submit transfer	**Transfer Complete!** shown without touching real backend.
`TC-TRF-MCK-002`	Transfer – 400 Bad Request	Resilience	P2	transfer.spec.ts	`status: 400`	Submit transfer	UI displays error panel (**An internal error has occurred**). No false success.
`TC-TRF-MCK-003`	Transfer – 401 Unauthorized	Resilience / Security	P2	transfer.spec.ts	`status: 401`	Submit transfer	UI displays error or prompts re-login. No confirmation shown.
`TC-TRF-MCK-004`	Transfer – 403 Forbidden	Resilience / Security	P3	transfer.spec.ts	`status: 403`	Submit transfer	UI displays permission error. No confirmation shown.
`TC-TRF-MCK-005`	Transfer – 404 Endpoint Not Found	Resilience	P3	transfer.spec.ts	`status: 404`	Submit transfer	Error handled gracefully. No blank page.
`TC-TRF-MCK-006`	Transfer – 500 Internal Server Error	Resilience	P1	transfer.spec.ts	`status: 500`	Submit transfer	Error panel visible. **Transfer Complete!** is not visible. No unhandled page crash.
`TC-TRF-MCK-007`	Transfer – Network Timeout	Resilience	P2	transfer.spec.ts	`route.abort('timedout')`	Submit transfer	Timeout handled cleanly; application remains responsive.
`TC-TRF-MCK-008`	Transfer – Empty Response Body	Resilience	P3	transfer.spec.ts	`status: 200, body: ''`	Submit transfer	Safe fallback handled; no undefined fields rendered.
`TC-TRF-MCK-009`	Transfer – Malformed JSON Response	Resilience	P3	transfer.spec.ts	`status: 200, body: '{"error":'`	Submit transfer	No unhandled JS exception (`pageerror` event does not fire).
`TC-ACC-MCK-001`	Accounts Overview – 500 Server Error	Resilience	P2	accounts.spec.ts	Route `**/customers/*/accounts` ➔ `status: 500`	Open Accounts Overview	Error message displayed instead of broken table.
`TC-ACC-MCK-002`	Accounts Overview – Empty Accounts Array	Resilience	P3	accounts.spec.ts	Route ➔ `status: 200, body: '[]'`	Open Accounts Overview	Table displays zero rows cleanly with $0.00 Total.

ID	Test Case	Classification	Priority	Target Spec	Mock Setup	Action	Expected Result
`TC-ACC-MCK-003`	Accounts Overview – Stubbed Balances	Resilience	P3	accounts.spec.ts	Route ➔ JSON with balances 100.00 & 250.50	Open Accounts Overview	Table renders exactly $100.00 and $250.50; Total is $350.50.
`TC-BILL-MCK-001`	Bill Pay – 500 Server Error	Resilience	P2	billPayment.spec.ts	Route `**/billpay*` ➔ `status: 500`	Submit bill payment	Error panel displayed. No confirmation rendered.
`TC-TXN-MCK-001`	Find Transactions – 500 Server Error	Resilience	P3	transactions.spec.ts	Route `**/transactions/**` ➔ `status: 500`	Search by amount	Error notification shown cleanly.

---

4. Overall Test Case Metrics

Automation Phase	Test Focus	Total Cases	Share of Suite
Phase 1: UI Testing	Forms, navigation, validation, user flows	**56**	~34%
Phase 2: API Testing	Status codes, schemas, business math, security	**75**	~45%
Phase 3: Hybrid & Mocking	E2E workflows, backend verification, resilience	**34** (20 Hybrid + 14 Mock)	~21%
Total		**165**	**100%**